Module 2: Control Flow

Control flow decides which lines run, and how many times. Master this module and you can write real logic. Type each program yourself, run it, then change the values.

What You Will Learn

Topic	Key ideas
2.1 Conditional statements	`if`, `elif`, `else`, nesting, ternary, `match`
2.2 for loops	`range()`, iterating sequences, `enumerate`, `zip`
2.3 while loops	conditions, counters, sentinel loops, menus
2.4 break, continue, pass, else	changing loop behaviour
2.5 Nested loops and patterns	star, number and Pascal patterns

Golden rule: Python blocks are defined by **indentation** (4 spaces) after a colon `:`.

Topic 2.1: Conditional Statements

Notes

- Syntax: `if condition:` then optional `elif condition:` blocks then optional `else:`.
- Only the **first** true branch runs; the rest are skipped.
- Conditions can be any expression; Python evaluates **truthiness** (empty string, `0`, `[]`, `None` are false).
- **Ternary:** `value_if_true if condition else value_if_false`.
- **match-case** (Python 3.10+) handles multiple fixed choices; `_` is the default case.
- Nesting is allowed, but prefer `and` / `or` to keep code flat.

Programs

```
# 1. Simple if
temperature = 38
if temperature > 35:
    print("It is hot today")
print("Program finished")
```

```
# 2. if-else: voting eligibility
age = int(input("Enter your age: "))
if age >= 18:
    print("You can vote")
else:
    print(f"You can vote after {18 - age} more year(s)")
```

```
# 3. if-elif-else: grade calculator
marks = int(input("Enter marks (0-100): "))
if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 40:
    grade = "C"
else:
    grade = "Fail"
print("Grade:", grade)
```

```
# 4. Nested if: simple login check
username = input("Username: ")
password = input("Password: ")
if username == "admin":
    if password == "1234":
        print("Welcome, admin")
    else:
        print("Wrong password")
else:
    print("Unknown user")
```

```
# 5. Ternary (conditional expression)
n = int(input("Enter a number: "))
parity = "even" if n % 2 == 0 else "odd"
print(n, "is", parity)
sign = "positive" if n > 0 else "negative" if n < 0 else "zero"
print(n, "is", sign)
```

```
# 6. Largest of three numbers
a, b, c = 14, 29, 21
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print("Largest:", largest)
print("Using max():", max(a, b, c))
```

```
# 7. Leap year checker
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

```
# 8. Truthiness in conditions
name = ""
items = [3, 5]
if not name:
    print("Name is empty")
if items:
    print("Items found:", items)
value = None
if value is None:
    print("No value set")
```

```
# 9. match-case (Python 3.10+)
day = int(input("Day number (1-7): "))
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3 | 4 | 5:
        print("Midweek")
    case 6 | 7:
        print("Weekend")
    case _:
        print("Invalid day")
```

```
# 10. Calculator using if-elif
a = float(input("First number: "))
op = input("Operator (+, -, *, /): ")
b = float(input("Second number: "))
if op == "+":
    print(a + b)
elif op == "-":
    print(a - b)
elif op == "*":
    print(a * b)
elif op == "/":
    print(a / b if b != 0 else "Cannot divide by zero")
else:
    print("Invalid operator")
```

Topic 2.2: for Loops

Notes

- `for item in iterable:` runs the block once per item (strings, lists, tuples, dicts, `range`, ...).
- `range(stop)`, `range(start, stop)`, `range(start, stop, step)`; `stop` is **excluded**.
- `range(10, 0, -1)` counts down.
- `enumerate(seq)` gives `(index, item)` pairs; `zip(a, b)` walks two sequences together.

- Use `for` when you know what you are looping over; use `while` when you loop until a condition changes.

Programs

```
# 1. range() basics
for i in range(5):
    print(i, end=" ")        # 0 1 2 3 4
print()
for i in range(1, 6):
    print(i, end=" ")        # 1 2 3 4 5
print()
```

```
# 2. Step and reverse
for i in range(0, 21, 5):
    print(i, end=" ")        # 0 5 10 15 20
print()
for i in range(10, 0, -2):
    print(i, end=" ")        # 10 8 6 4 2
print()
```

```
# 3. Looping over a string and a list
for ch in "Python":
    print(ch, end="-")
print()
for fruit in ["apple", "banana", "mango"]:
    print(fruit.upper())
```

```
# 4. Sum and average of 1 to n
n = int(input("Enter n: "))
total = 0
for i in range(1, n + 1):
    total += i
print("Sum =", total)
print("Average =", total / n)
```

```
# 5. Multiplication table
n = int(input("Table of: "))
for i in range(1, 11):
    print(f"{n} x {i:>2} = {n * i}")
```

```
# 6. Factorial
n = int(input("Enter n: "))
fact = 1
for i in range(2, n + 1):
    fact *= i
print(f"{n}! = {fact}")
```

```
# 7. enumerate() and zip()
names = ["Asha", "Ravi", "Meena"]
marks = [88, 72, 95]
for index, name in enumerate(names, start=1):
    print(index, name)
for name, mark in zip(names, marks):
    print(f"{name}: {mark}")
```

```
# 8. Count vowels in a string
text = input("Enter text: ").lower()
count = 0
for ch in text:
    if ch in "aeiou":
        count += 1
print("Vowels:", count)
```

```
# 9. Fibonacci series
n = int(input("How many terms? "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

```
# 10. Even numbers, sum and manual max of a list
nums = [12, 7, 45, 3, 28, 19]
evens = []
biggest = nums[0]
for x in nums:
    if x % 2 == 0:
        evens.append(x)
    if x > biggest:
        biggest = x
print("Evens:", evens)
print("Sum:", sum(nums), "| Max:", biggest)
```

Topic 2.3: while Loops

Notes

- `while condition:` repeats as long as the condition is true. Make sure something inside the loop **changes** the condition, or it never ends.
- Typical pieces: initialise a counter, test it, update it.
- `while True:` plus `break` is the standard pattern for menus and input validation.
- Press **Ctrl+C** to stop an infinite loop in the terminal.
- Digit tricks: `n % 10` is the last digit, `n // 10` removes it.

Programs

```
# 1. Counting
i = 1
while i <= 5:
    print("Count:", i)
    i += 1
```

```
# 2. Sum of digits
n = int(input("Enter a number: "))
total = 0
while n > 0:
    total += n % 10
    n //= 10
print("Sum of digits =", total)
```

```
# 3. Reverse a number
n = int(input("Enter a number: "))
rev = 0
while n > 0:
    rev = rev * 10 + n % 10
    n //= 10
print("Reversed =", rev)
```

```
# 4. Password with limited attempts
attempts = 0
while attempts < 3:
    pwd = input("Password: ")
    if pwd == "python123":
        print("Access granted")
        break
    attempts += 1
    print(f"Wrong. {3 - attempts} attempt(s) left")
```

```
# 5. Number guessing game
import random
secret = random.randint(1, 20)
tries = 0
while True:
    guess = int(input("Guess (1-20): "))
    tries += 1
    if guess < secret:
        print("Too low")
    elif guess > secret:
        print("Too high")
    else:
        print(f"Correct in {tries} tries!")
        break
```

```
# 6. Factorial using while
n = int(input("Enter n: "))
fact, i = 1, 1
while i <= n:
    fact *= i
    i += 1
print(f"{n}! = {fact}")
```

```
# 7. Collatz sequence
n = 27
steps = 0
while n != 1:
    n = n // 2 if n % 2 == 0 else 3 * n + 1
    steps += 1
print("Reached 1 in", steps, "steps")
```

```
# 8. Palindrome number check
n = int(input("Enter a number: "))
original, rev = n, 0
while n > 0:
    rev = rev * 10 + n % 10
    n //= 10
print("Palindrome" if original == rev else "Not a palindrome")
```

```
# 9. Menu-driven program
balance = 1000
while True:
    print("\n1. Check balance  2. Deposit  3. Withdraw  4. Exit")
    choice = input("Choose: ")
    if choice == "1":
        print("Balance:", balance)
    elif choice == "2":
        balance += float(input("Amount: "))
    elif choice == "3":
        amt = float(input("Amount: "))
        if amt <= balance:
            balance -= amt
        else:
            print("Insufficient funds")
    elif choice == "4":
        print("Goodbye")
        break
    else:
        print("Invalid choice")
```

```
# 10. GCD using Euclid's algorithm and binary conversion
a, b = 48, 18
while b != 0:
    a, b = b, a % b
print("GCD =", a)
n = 25
bits = ""
while n > 0:
    bits = str(n % 2) + bits
    n //= 2
print("Binary of 25 =", bits)
```

Topic 2.4: break, continue, pass and loop else

Notes

Keyword	Effect
`break`	Exit the loop immediately
`continue`	Skip the rest of this iteration, go to the next
`pass`	Do nothing (placeholder for code you will write later)
`else` on a loop	Runs only if the loop finished **without** `break`

- `break` and `continue` affect only the **innermost** loop.
- Loop `else` is perfect for "search, and tell me if nothing was found".

Programs

```
# 1. break: stop at the first match
for n in [4, 9, 15, 22, 30]:
    if n % 5 == 0:
        print("First multiple of 5:", n)
        break
```

```
# 2. continue: skip even numbers
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i, end=" ")
print()
```

```
# 3. pass as a placeholder
for i in range(3):
    pass                # TODO: add logic later
def not_ready_yet():
    pass
if True:
    pass
print("No errors, nothing happened")
```

```
# 4. for-else: search with a "not found" message
names = ["Asha", "Ravi", "Meena"]
target = input("Search name: ")
for name in names:
    if name == target:
        print("Found!")
        break
else:
    print("Not found")
```

```
# 5. while-else
n = 0
while n < 3:
    print("n =", n)
    n += 1
else:
    print("Loop ended normally")
```

```
# 6. Prime check using for-else
n = int(input("Enter a number: "))
if n < 2:
    print("Not prime")
else:
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            print("Not prime, divisible by", i)
            break
    else:
        print("Prime")
```

```
# 7. continue: sum only positive numbers
data = [5, -3, 8, -1, 0, 12]
total = 0
for x in data:
    if x <= 0:
        continue
    total += x
print("Sum of positives:", total)
```

```
# 8. Sentinel value: stop when the user types 'done'
total = 0
while True:
    entry = input("Enter a number (or 'done'): ")
    if entry == "done":
        break
    total += float(entry)
print("Total =", total)
```

```
# 9. FizzBuzz using continue
for i in range(1, 21):
    if i % 15 == 0:
        print("FizzBuzz")
        continue
    if i % 3 == 0:
        print("Fizz")
        continue
    if i % 5 == 0:
        print("Buzz")
        continue
    print(i)
```

```
# 10. Breaking out of nested loops with a flag
found = False
for i in range(1, 10):
    for j in range(1, 10):
        if i * j == 42:
            print("Found:", i, "x", j)
            found = True
            break            # leaves the inner loop only
    if found:
        break                # leaves the outer loop
```

Topic 2.5: Nested Loops and Patterns

Notes

- A loop inside a loop: for each iteration of the **outer** loop, the **inner** loop runs fully.
- Outer loop usually controls **rows**, inner loop controls **columns**.
- `print(..., end="")` stays on the same line; a bare `print()` moves to the next line.
- Useful trick: `" " * k` for spaces and `"*" * k` for stars.
- Row `i` of a pyramid with `n` rows has `n - i` spaces and `2*i - 1` stars.

Programs

```
# 1. Rectangle of stars
rows, cols = 4, 6
for i in range(rows):
    for j in range(cols):
        print("*", end=" ")
    print()
```

```
# 2. Right triangle
n = 5
for i in range(1, n + 1):
    print("* " * i)
```

```
# 3. Inverted triangle
n = 5
for i in range(n, 0, -1):
    print("* " * i)
```

```
# 4. Pyramid
n = 5
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

```
# 5. Number triangle
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

```
# 6. Diamond
n = 4
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
for i in range(n - 1, 0, -1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

```
# 7. Hollow square
n = 5
for i in range(n):
    for j in range(n):
        if i == 0 or i == n - 1 or j == 0 or j == n - 1:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

```
# 8. Multiplication table grid
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i * j:>4}", end="")
    print()
```

```
# 9. Floyd's triangle
n = 5
num = 1
for i in range(1, n + 1):
    for j in range(i):
        print(num, end=" ")
        num += 1
    print()
```

```
# 10. Pascal's triangle (the list expression is a preview of Module 9)
n = 6
row = [1]
for _ in range(n):
    print(" ".join(map(str, row)).center(n * 4))
    row = [1] + [row[i] + row[i + 1] for i in range(len(row) - 1)] + [1]
```

Module 2 Practice Tasks

Solve these without looking at the programs above:

1. Read three side lengths and say whether they form a triangle, and if so which kind (equilateral, isosceles, scalene).
2. Print all numbers from 1 to 100 divisible by both 4 and 6.
3. Check whether a number is an Armstrong number (for example 153 = 1^3 + 5^3 + 3^3).

4. Print all prime numbers between 1 and 100.
5. Keep reading numbers until the user enters 0, then print the count, sum and average.
6. Reverse a string using a loop (no slicing).
7. Print the sum of the series 1 + 1/2 + 1/3 + ... + 1/n.
8. Build an ATM-style menu with a PIN allowing only 3 attempts.
9. Print this pattern for n = 5: each row `i` prints the letter `A`, `B`, `C`... repeated `i` times.
10. Print the first 10 numbers of the Collatz sequence starting from a number the user enters.

Module 2 Cheat Sheet

```
if cond1:  ...
elif cond2: ...
else:      ...

x = a if cond else b              # ternary
match v:
    case 1 | 2: ...
    case _:     ...

for i in range(start, stop, step): ...
for i, item in enumerate(seq, start=1): ...
for a, b in zip(list1, list2): ...

while cond: ...                   # update the condition inside!
break      # leave the loop
continue   # next iteration
pass       # do nothing
for/while ... else: ...           # else runs if no break happened
```

Common mistakes: forgetting the colon, wrong indentation, an infinite `while` loop with no update, off-by-one errors with `range` (stop is excluded), and using `=` instead of `==` in conditions.